Passagem de projeto — ENCIVIL Gestão

Guia curto para quem continua o desenvolvimento (revisto em 2026-11-12). Lê isto primeiro; depois o `CLAUDE.md` (regras completas).
Índice de toda a documentação: `docs/README.md`.

1. O que é

ERP interno da ENCIVIL (construção civil, Portugal). **Está em produção** e é usado todos os dias.

Módulo	Para quê
Armazém	Artigos, stock, entradas/saídas por obra, ferramentas, inventário
Obras	Ficha da obra, relatórios diários, subempreitadas, autos de medição, orçamento e custos
Frota	Viaturas e máquinas, entregas, manutenção, revisões
Abastecimento	Pedido de combustível pelo motorista (QR), aprovação, bomba Polo 2, leitura do talão com IA
RH	Colaboradores, horários, faltas, picagens com geolocalização, EPIs, formações, alertas
Faturas e contabilidade	Extração de faturas com IA, exportação para o contabilista
Gestão	Início por cargo, relatórios, auditoria, utilizadores (com MFA), backup

- Site: https://app.encivilconstroi.com
- Código: GitHub `Encivil-Constroi/encivil-gestao`, branch `main`.

2. Stack

Camada	Tecnologia
Frontend	React 18, TypeScript (strict), Vite, Tailwind CSS v4, shadcn/ui
Backend	Supabase: PostgreSQL, Auth, Storage, RLS, Edge Functions (Deno)
Alojamento	Cloudflare Pages (cada push para `main` publica sozinho)
Telemóvel	PWA (funciona como app, aguenta sem rede)
Erros	Sentry
Testes	Vitest (frontend) + PGlite (Postgres em memória para testar a BD)

3. Primeiro dia

1. Pede acesso às contas (GitHub, Supabase, Cloudflare). A lista está em `docs/13-infraestrutura-e-contas.md`.

2. Instala o Node 24, o Git e (para o ambiente local, secção 9) o Docker Desktop.
3. Na pasta do projeto:

```
npm install
cp .env.example .env.local   # pede os valores ao responsável; nunca os ponhas no Git
npm run dev                  # http://localhost:5173
```

> ⚠️ `npm run dev` **liga à base de dados de PRODUÇÃO**, com dados e utilizadores reais (também sem `.env.local`). Usa-o só para ver. Não cries, alteres nem apagues registos para testar.

4. Onde está cada coisa

```
src/app/pages/        ecrãs que juntam módulos (ex.: armazem/ = produtos + movimentos +
ferramentas)
src/app/components/   componentes partilhados (PageHeader, FiltroChips, CampoSelect…)
src/app/routes.tsx    todas as rotas
src/features/<m>/     um módulo por pasta (obras, frota, colaboradores, produtos, movimentos…)
src/styles/theme.css  cores e identidade visual (usa só estes tokens)
supabase/migrations/  todas as alterações à base de dados (SQL, 101 ficheiros em 2026-11-12)
supabase/functions/   Edge Functions
supabase/tests/       testes da base de dados
src/__tests__/        testes do frontend
docs/                 documentação (índice em docs/README.md)
scripts/carga/        dados fictícios, testes de carga e concorrência (só ambiente local)
```

5. Comandos

```
npm run dev          # app em modo de desenvolvimento
npm run typecheck    # tem de dar 0 erros
npm run lint         # sem avisos (o CI também corre)
npm test             # todos os testes (frontend + base de dados)
npm run build        # gera a versão de produção
npm run check:edge && npm run test:edge   # Edge Functions
```

O Git corre verificações sozinho: no commit (procura segredos e faz o typecheck) e no push (faz o build). **Nunca uses `--no-verify`.**

6. Como desenvolver uma alteração

1. Cria um branch: `git checkout -b feat/nome-curto`.
2. Lê a doc do módulo em `docs/` (a tabela "Módulos" do `CLAUDE.md` diz qual).
3. Escreve primeiro o teste que falha, depois o código.
4. Corre `npm run typecheck`, `npm run lint`, `npm test` e `npm run build`.
5. Commit em português: `fix(obras): …`, `feat(frota): …`.
6. Merge para `main` e push. O Cloudflare publica o site sozinho em poucos minutos.

Regras que não se quebram

- **Stock** só muda por movimentos (RPC `registar_movimento`). Nunca editar `stock_atual` à mão. Nunca apagar nem editar `movimentos_stock`.
- **Segurança fica na base de dados** (RLS e funções). Esconder um botão na interface não protege nada.
- **Cada tabela ou função nova** precisa de `GRANT` + `ENABLE ROW LEVEL SECURITY` + policies.
- **Cores** só de `theme.css`. Nada de cores fixas.
- **Páginas novas** usam os componentes partilhados (`PageHeader`, `FiltroChips`, `CampoSelect`…). Lista no `CLAUDE.md` › Identidade visual.
- **Listas grandes**: o Supabase corta em 1000 linhas sem avisar. Para trazer tudo usa `buscarTodas` (`src/app/lib/buscarTodas.ts`).
- **Texto da app** em português de Portugal.
- **Chaves secretas** (`sb_secret_*`) nunca no frontend nem no Git.

7. Alterar a base de dados

1. Cria um ficheiro em `supabase/migrations/AAAAMMDDHHMMSS_nome.sql` (data maior que a última).
2. O SQL tem de poder correr duas vezes sem erro e acaba com um bloco `-- ROLLBACK` comentado.
3. Escreve um teste em `supabase/tests/` (copia um parecido).
4. **Publicação, sempre por esta ordem:**
    1. o site (push para `main`);
    2. a migration, colada à mão no Supabase › SQL Editor, fora do horário de trabalho;
    3. as Edge Functions, se mudaram: `npx supabase functions deploy <nome>`. Depois confirma no Dashboard se a opção "Enforce JWT verification" ficou como em `supabase/config.toml` (`verify_jwt`).
5. Regenera os tipos (no Git Bash, não no PowerShell — o PowerShell grava o ficheiro em UTF-16):

```
npx supabase gen types typescript --project-id <id do projeto> >
src/integrations/supabase/types.ts
```

(com a stack local a correr, `--local` em vez de `--project-id`). `types.ts` é gerado: nunca o editar à mão.

8. Uso de IA no projeto

O projeto foi desenvolvido com ajuda do **Claude Code** (assistente de programação da Anthropic) e está preparado para continuar assim.

- **`CLAUDE.md`**: as regras do projeto. A IA lê-o sozinha em cada sessão. Se mudares uma regra importante, atualiza-o.
- **`AGENTS.md`**: o mesmo resumo, para outras ferramentas de IA. Tem de dizer o mesmo que o `CLAUDE.md`.
- **Skills em `.agents/skills/`**: formas de trabalhar que a IA segue, como planear antes de mexer, testar primeiro, rever o código e verificar antes de dizer "feito".
- **Comandos em `.claude/commands/`**:
    - `/deploy`: publica o site.
    - `/migration`: cria uma migration da forma segura.
    - `/review`: revê o código segundo as regras do projeto.
- **`agent-browser`**: a IA abre a app no browser para testar ecrãs.

Como pedir bem à IA

- Diz o objetivo e o módulo ("na ficha da obra, quero…").
- Pede **plano primeiro** em mudanças grandes e aprova-o antes de ela executar.
- Exige provas: typecheck, testes e build a verde.

Cuidados com a IA

- A IA **não aplica** migrations nem publica Edge Functions sozinha: faz isso tu.
- Para a IA testar com dados, usa o **ambiente local** (secção 9), nunca produção.
- Lê sempre o resumo do que ela fez antes de fazer merge.

9. Ambiente local seguro (para testes a sério)

Para testar sem tocar em produção: Supabase local em Docker, com as migrations aplicadas e dados fictícios.

```
npx supabase start          # precisa do Docker Desktop; mostra a URL e as chaves locais
# no Git Bash, com a chave sb_publishable_* que o comando mostra:
VITE_SUPABASE_URL=http://127.0.0.1:54321 VITE_SUPABASE_PUBLISHABLE_KEY=sb_publishable_... npx
vite
```

- Se a configuração local estiver errada, a app **dá erro em vez de ir para produção** (proteção em `src/integrations/supabase/config.ts`).
- Dados fictícios e testes de carga: `scripts/carga/` (lê o `README.md` dessa pasta). Os scripts recusam qualquer alvo que não seja a tua máquina. Atenção: exigem `CARGA_WORKDIR` a apontar para uma cópia da stack com `project_id = "encivil-checkup"`, não a da raiz do repositório.

10. Estado atual e pendentes

- Último checkup completo de segurança, bugs e carga: `docs/auditorias/2026-10-28-checkup-geral.md`.
- Polimento visual de novembro de 2026: `docs/superpowers/plans/2026-11-02-design-polimento.md`.
- Ideias para o futuro: `docs/melhorias-futuras.html` e `docs/12-plano-v3.md`.

Pendentes conhecidos:

- As migrations do checkup (`20261028*`) e as Edge Functions foram aplicadas em produção em 2026-10-29. Estado de cada migration de outubro e script para confirmar: `docs/22-seguranca-operacao.md`.
- Faturas lançadas antes da correção do checkup (2026-10-28) podem ter o stock em dobro (duplo clique). Convém revê-las.
- Ligar o MFA obrigatório só depois de os aprovadores de combustível o registarem.
- Decidir se o papel `armazem` pode fazer contagem de inventário.
- Branches antigos já integrados em `main` (`checkup/2026-10-28`, `chore/tipos-checkup`, `design/polimento-2026-11`, `feat/controlo-subempreitadas`, `seguranca-2026`, `seg-*`) podem ser apagados. Os `seg-*` e `seguranca-2026` estão abertos em worktrees (`C:/wt/...`): primeiro `git worktree remove`, e só depois de confirmares que não há nada por guardar e que o `node_modules` dessa pasta não é uma junção para o do projeto (apagá-la apagaria o verdadeiro).
- O remote `antigo-vercel` é legado: **nunca dar push para lá**.

11. Documentos para ler a seguir

Documento	Porquê
`CLAUDE.md`	Todas as regras do projeto
`docs/README.md`	O que é cada documento e se está atual
`docs/02-regras-de-negocio.md`	Regras de stock e movimentos
`docs/09-implantacao.md`	Como publicar
`docs/22-seguranca-operacao.md`	Backups, MFA, recuperação
`docs/13-infraestrutura-e-contas.md`	Contas e quem é dono de quê
`ARCHITECTURE.md`	Visão geral da arquitetura